

Leaflet for the map: panning, zoom, markers.

CARTO raster tiles for the map layer.

Labels are a separate layer, drawn on top.



A waypoint is a **lat/lng** pair.

?w=40.7391,-73.9905&vt=Add+bike+lane

Each point snaps to the nearest **graph node**.

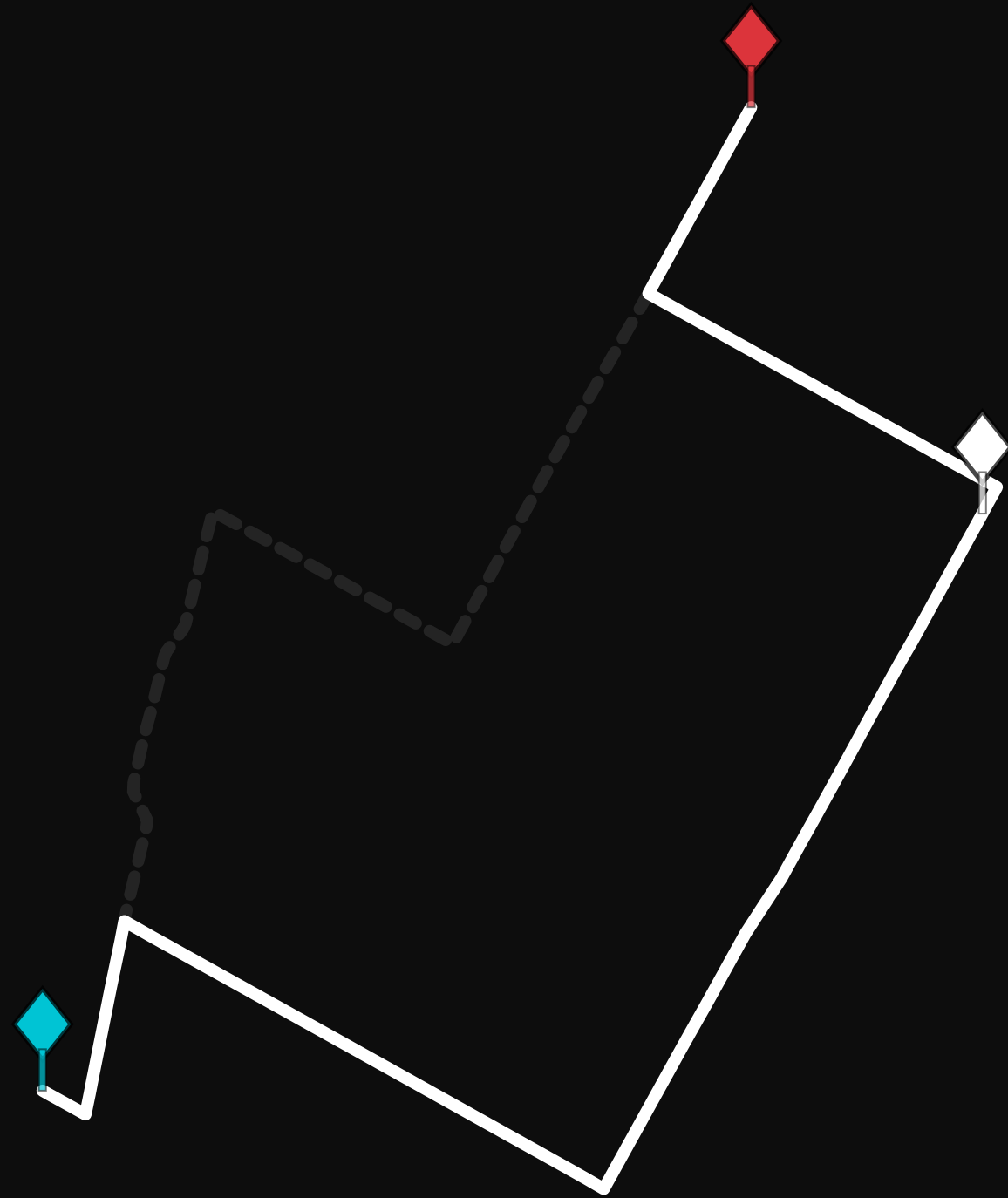


Two waypoints, one route.

OSRM does the routing with
Multilevel Dijkstra.



Drag the line to add a **midwaypoint**.
Tap a waypoint to **delete** it.
First is **start**, last is **end**.
Everything in between is a midwaypoint.



Casts get sent to Flask via a **POST**

Then Flask writes it, claims a **revision**,
and publishes one **delta** to Redis

Then all **Flask instances** subscribe to Redis
and get the revision and delta

Then they synchronize to each **client tab**
via a **websocket** connection



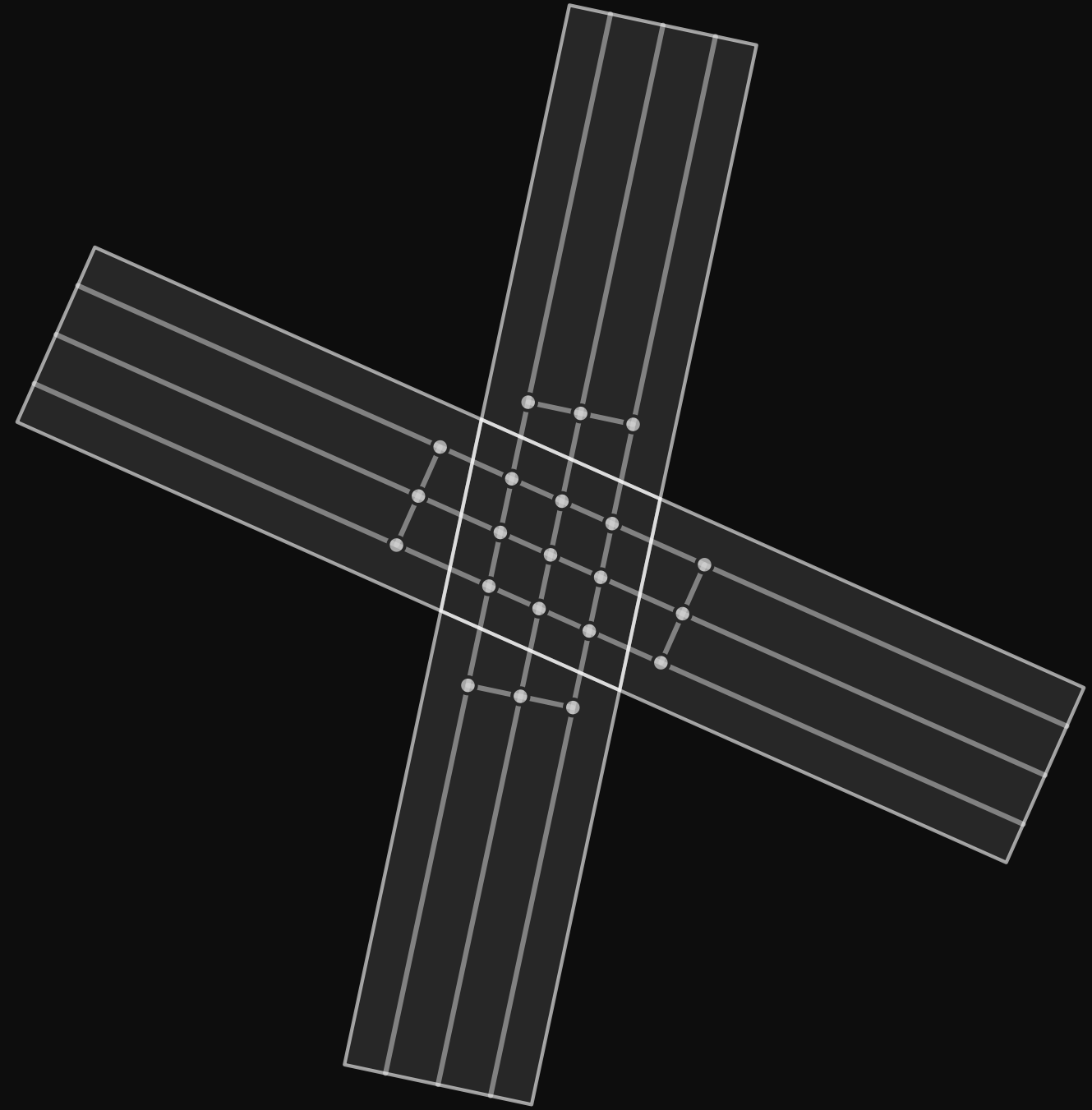
```
{
  "type": "delta",    "rev": 8241,
  "m": "walkways",   "vt": 37,
  "dir": 1,           "reversed": false,
  "vtLabel": "Add bike lane",
  "edges": [10432, 10433],
  "vtCounts": { "10432": [12, 1] },
  "blockCounts": { "884": [9, 1] }
}
```

One crossing becomes **5 blocks**:

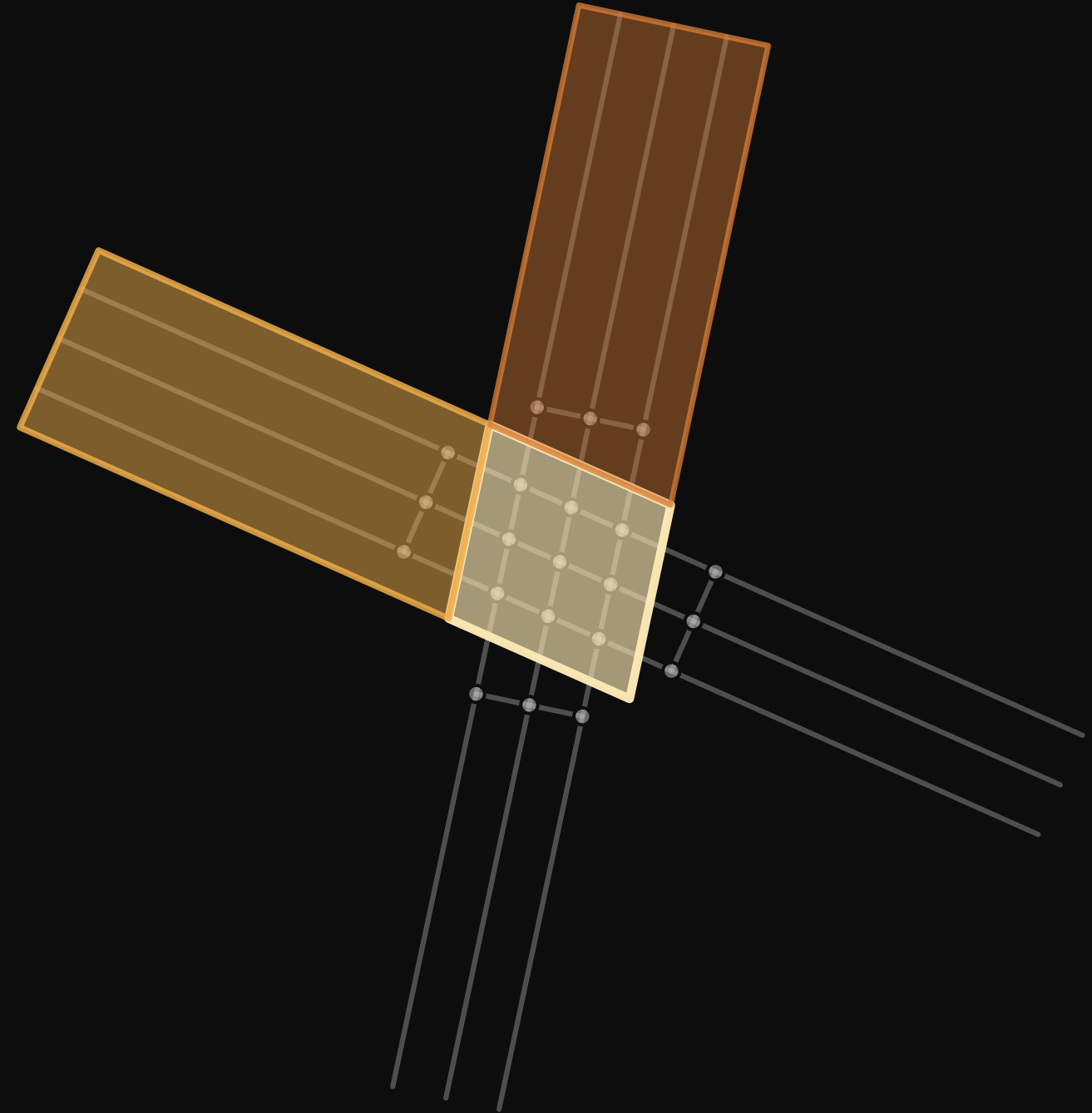
4 street blocks + 1 intersection.

Each graph edge/node is assigned
to exactly one block.

Each device gets **one vote per block, per type.**



Block heat = **(up votes) - (down votes)**.
Log-scaled and min-max-scaled.

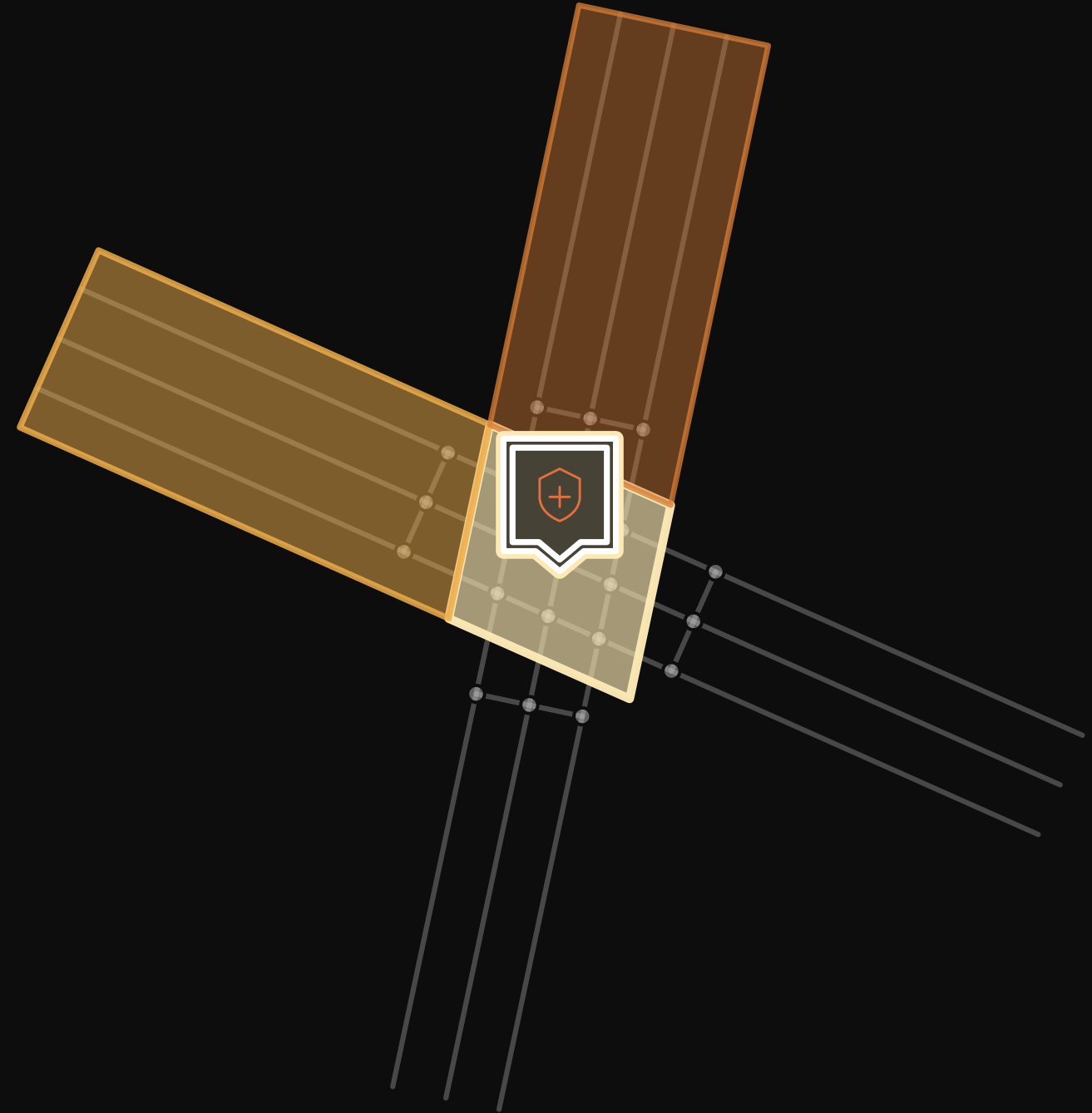


Top Point Proposals

For each vote type, the **5 highest net vote blocks** are "top proposals".

Same-type pins must be **>1 km** apart.

50 pins per map.



Top Route Proposals

Start at the **highest vote count edge**.

Grow along the **best neighbor**, both ways.

Pin a ghost waypoint where routing diverges.

Length budget: **$2700 \text{ m} + 660 \cdot \sqrt{\text{score}}$**

(after some tweaking).

No more than 5 waypoints.

